

JAVA PROGRAMMING

QUESTION:

Design and implement a console-based Timesheet & Payroll application to log hours, approve timesheets, and compute payroll using OOP in Java. Requirements: 1. Create at least 4 classes:

- Employee – empId, name, role, hourlyRate, bankAccount.
- Project – projectId, name, client, billableRate.
- Timesheet – sheetId, empId, entries, weekStart, state.
- PayrollService – approve sheets, compute pay, generate slips.

FOLDER STRUCTURE:

```
src/
├── Employee.java
├── Manager.java
├── Project.java
├── Timesheet.java
├── PayrollService.java
└── TimePayrollMain.java
```

JAVA IMPLEMENTATION:

Employee.java:

```
package jeganath;
public class Employee {
    protected int empId;
    protected String name;
    protected String role;
    protected double hourlyRate;
    protected String bankAccount;
    public Employee(int empId, String name, String role, double hourlyRate, String
bankAccount) {
        this.empId = empId;
        this.name = name;
        this.role = role;
        this.hourlyRate = hourlyRate;
        this.bankAccount = bankAccount;
    }
    // Getters & Setters
    public int getEmpId() { return empId; }
    public String getName() { return name; }
    public String getRole() { return role; }
    public double getHourlyRate() { return hourlyRate; }
    public String getBankAccount() { return bankAccount; }
    public void setHourlyRate(double hourlyRate) { this.hourlyRate = hourlyRate; }
    // Method to compute pay (base logic)
    public double computePay(int totalHours) {
        return totalHours * hourlyRate;
    }
    @Override
    public String toString() {
        return empId + " - " + name + " (" + role + ")";
    }
}
```

```
package jeganath;

public class Employee {

    protected int empId;

    protected String name;

    protected String role;

    protected double hourlyRate;

    protected String bankAccount;

    public Employee(int empId, String name, String role, double hourlyRate, String
bankAccount) {

        this.empId = empId;

        this.name = name;

        this.role = role;

        this.hourlyRate = hourlyRate;

        this.bankAccount = bankAccount;

    }

    // Getters & Setters

    public int getEmpId() { return empId; }

    public String getName() { return name; }

    public String getRole() { return role; }

    public double getHourlyRate() { return hourlyRate; }

    public String getBankAccount() { return bankAccount; }

    public void setHourlyRate(double hourlyRate) { this.hourlyRate = hourlyRate; }

    // Method to compute pay (base logic)

    public double computePay(int totalHours) {

        return totalHours * hourlyRate;

    }

    @Override

    public String toString() {

        return empId + " - " + name + " (" + role + ")";

    }

}
```

Manager.java:

```
package jeganath;
public class Manager extends Employee {
    public Manager(int empId, String name, double hourlyRate, String bankAccount) {
        super(empId, name, "Manager", hourlyRate, bankAccount);
    }
    // Method overriding → with overtime pay
    @Override
    public double computePay(int totalHours) {
        double basePay = totalHours * hourlyRate;
        if (totalHours > 40) {
            basePay += (totalHours - 40) * hourlyRate * 0.5; // Overtime bonus
        }
        return basePay;
    }
    // Manager approves a timesheet
    public void approveTimesheet(Timesheet sheet) {
        sheet.setState("APPROVED");
        System.out.println("Manager approved Timesheet ID: " + sheet.getSheetId());
    }
}
```

```
package jeganath;
```

```
public class Manager extends Employee {
```

```
    public Manager(int empId, String name, double hourlyRate, String bankAccount) {
```

```
        super(empId, name, "Manager", hourlyRate, bankAccount);
```

```
    }
```

```
    // Method overriding → with overtime pay
```

```
    @Override
```

```
    public double computePay(int totalHours) {
```

```
        double basePay = totalHours * hourlyRate;
```

```
        if (totalHours > 40) {
```

```
            basePay += (totalHours - 40) * hourlyRate * 0.5; // Overtime bonus
```

```
        }
```

```
        return basePay;
```

```
    }
```

```
    // Manager approves a timesheet
```

```
    public void approveTimesheet(Timesheet sheet) {
```

```
        sheet.setState("APPROVED");
```

```
        System.out.println("Manager approved Timesheet ID: " + sheet.getSheetId());
```

```
    }
```

```
}
```

Project.java:

```
package jeganath;
public class Project {
    private int projectId;
    private String name;
    private String client;
    private double billableRate;
    public Project(int projectId, String name, String client, double billableRate) {
        this.projectId = projectId;
        this.name = name;
        this.client = client;
        this.billableRate = billableRate;
    }
    // Getters
    public int getProjectId() { return projectId; }
    public String getName() { return name; }
    public String getClient() { return client; }
    public double getBillableRate() { return billableRate; }
    @Override
    public String toString() {
        return "Project " + name + " for " + client;
    }
}
```

```
package jeganath;

public class Project {

    private int projectId;

    private String name;

    private String client;

    private double billableRate;

    public Project(int projectId, String name, String client, double billableRate) {

        this.projectId = projectId;

        this.name = name;

        this.client = client;

        this.billableRate = billableRate;

    }

    // Getters

    public int getProjectId() { return projectId; }

    public String getName() { return name; }

    public String getClient() { return client; }

    public double getBillableRate() { return billableRate; }

    @Override

    public String toString() {
```

```

        return "Project " + name + " for " + client;
    }
}

```

Timesheet.java:

```

package jeganath;
import java.util.HashMap;
import java.util.Map;
public class Timesheet {
    private int sheetId;
    private int empId;
    private String weekStart;
    private String state; // PENDING / APPROVED
    private Map<String, Integer> entries; // date → hours
    private Project project; // link timesheet to project
    public Timesheet(int sheetId, int empId, String weekStart, Project project) {
        this.sheetId = sheetId;
        this.empId = empId;
        this.weekStart = weekStart;
        this.state = "PENDING";
        this.entries = new HashMap<>();
        this.project = project;
    }
    // Method Overloading - logHours
    public void logHours(String date, int hours) {
        entries.put(date, hours);
    }
    public void logHours(String date, int startHour, int endHour) {
        int hours = endHour - startHour;
        entries.put(date, hours);
    }
    public int getTotalHours() {
        return entries.values().stream().mapToInt(Integer::intValue).sum();
    }
    public int getSheetId() { return sheetId; }
    public int getEmpId() { return empId; }
}

```

```

    public int getSheetId() { return sheetId; }
    public int getEmpId() { return empId; }
    public String getState() { return state; }
    public void setState(String state) { this.state = state; }
    public Project getProject() { return project; }
    public void printEntries() {
        System.out.println("Timesheet Entries for " + project.getName() + ":");
        for (Map.Entry<String, Integer> e : entries.entrySet()) {
            System.out.println(e.getKey() + " -> " + e.getValue() + " hrs");
        }
    }
}

```

```

package jeganath;

import java.util.HashMap;

import java.util.Map;

public class Timesheet {

    private int sheetId;

    private int empId;

    private String weekStart;

    private String state; // PENDING / APPROVED

```

```

private Map<String, Integer> entries; // date → hours
private Project project; // link timesheet to project
public Timesheet(int sheetId, int empId, String weekStart, Project project) {
    this.sheetId = sheetId;
    this.empId = empId;
    this.weekStart = weekStart;
    this.state = "PENDING";
    this.entries = new HashMap<>();
    this.project = project;
}
// Method Overloading - logHours
public void logHours(String date, int hours) {
    entries.put(date, hours);
}
public void logHours(String date, int startHour, int endHour) {
    int hours = endHour - startHour;
    entries.put(date, hours);
}
public int getTotalHours() {

    return entries.values().stream().mapToInt(Integer::intValue).sum();
}
public int getSheetId() { return sheetId; }
public int getEmpId() { return empId; }
public String getState() { return state; }
public void setState(String state) { this.state = state; }
public Project getProject() { return project; }
public void printEntries() {
    System.out.println("Timesheet Entries for " + project.getName() + ":");
    for (Map.Entry<String, Integer> e : entries.entrySet()) {
        System.out.println(e.getKey() + " -> " + e.getValue() + " hrs");
    }
}
}

```

```
}
```

PayrollService.java:

```
package jeganath;
public class PayrollService {
    // Compute pay for an employee based on timesheet
    public void computePay(Employee emp, Timesheet sheet) {
        if (!sheet.getState().equals("APPROVED")) {
            System.out.println("Timesheet not approved. Cannot compute pay.");
            return;
        }
        int totalHours = sheet.getTotalHours();
        double pay = emp.computePay(totalHours);
        generateSlip(emp, sheet, totalHours, pay);
    }
    // Generate payslip
    public void generateSlip(Employee emp, Timesheet sheet, int hours, double pay) {
        System.out.println("\n---- Payslip ----");
        System.out.println("Employee: " + emp.getName());
        System.out.println("Role: " + emp.getRole());
        System.out.println("Project: " + sheet.getProject().getName());
        System.out.println("Total Hours: " + hours);
        System.out.println("Pay Amount: ₹" + pay);
        System.out.println("Bank Account: " + emp.getBankAccount());
        System.out.println("-----\n");
    }
    // Generate project-wise report
    public void projectReport(Timesheet sheet) {
        System.out.println("Project Report for " + sheet.getProject().getName());
        System.out.println("Client: " + sheet.getProject().getClient());
        System.out.println("Total Hours: " + sheet.getTotalHours());
        System.out.println("Billable Amount: ₹" + (sheet.getTotalHours() *
sheet.getProject().getBillableRate()));
        System.out.println("-----");
    }
    // Generate project-wise report
    public void projectReport(Timesheet sheet) {
        System.out.println("Project Report for " + sheet.getProject().getName());
        System.out.println("Client: " + sheet.getProject().getClient());
        System.out.println("Total Hours: " + sheet.getTotalHours());
        System.out.println("Billable Amount: ₹" + (sheet.getTotalHours() *
sheet.getProject().getBillableRate()));
        System.out.println("-----");
    }
}
```

```
package jeganath;
```

```
public class PayrollService {
```

```
    // Compute pay for an employee based on timesheet
```

```
    public void computePay(Employee emp, Timesheet sheet) {
```

```
        if (!sheet.getState().equals("APPROVED")) {
```

```
            System.out.println("Timesheet not approved. Cannot compute pay.");
```

```
            return;
```

```
        }
```

```
        int totalHours = sheet.getTotalHours();
```

```
        double pay = emp.computePay(totalHours);
```

```
        generateSlip(emp, sheet, totalHours, pay);
```

```

    }

    // Generate payslip

    public void generateSlip(Employee emp, Timesheet sheet, int hours, double pay) {

        System.out.println("\n---- Payslip---- ");

        System.out.println("Employee: " + emp.getName());

        System.out.println("Role: " + emp.getRole());

        System.out.println("Project: " + sheet.getProject().getName());

        System.out.println("Total Hours: " + hours);

        System.out.println("Pay Amount: ₹" + pay);

        System.out.println("Bank Account: " + emp.getBankAccount());

        System.out.println(" ----- \n");

    }

    // Generate project-wise report

    public void projectReport(Timesheet sheet) {

        System.out.println("Project Report for " + sheet.getProject().getName());

        System.out.println("Client: " + sheet.getProject().getClient());

        System.out.println("Total Hours: " + sheet.getTotalHours());

        System.out.println("Billable Amount: ₹" + (sheet.getTotalHours() *
sheet.getProject().getBillableRate()));

        System.out.println(" -----");

    }

}

```

TimePayrollMain.java:

```

package jeganath;
public class TimePayrollMain {
    public static void main(String[] args) {
        // Create Employees
        Employee e1 = new Employee(118, "JEGANATH", "Developer", 500, "AC12345");
        Manager m1 = new Manager(124, "MONISH", 800, "AC54321");
        // Create Project
        Project p1 = new Project(301, "Payroll System", "TechCorp", 1500);
        // Create Timesheet for JEGANATH
        Timesheet t1 = new Timesheet(401, e1.getEmpId(), "2025-09-01", p1);
        t1.logHours("2025-09-01", 8);
        t1.logHours("2025-09-02", 9);
        t1.logHours("2025-09-03", 10);
        t1.logHours("2025-09-04", 8);
        t1.logHours("2025-09-05", 7);
        // Print Entries
        t1.printEntries();
        // Manager Approves
        m1.approveTimesheet(t1);
        // Payroll Service
        PayrollService ps = new PayrollService();
        ps.computePay(e1, t1);
        ps.computePay(m1, t1);
        // Project Report
        ps.projectReport(t1);
    }
}

```



```
package jeganath;

public class TimePayrollMain {

    public static void main(String[] args) {

        // Create Employees

        Employee e1 = new Employee(118, "JEGANATH", "Developer", 500, "AC12345");
        Manager m1 = new Manager(124, "MONISH", 800, "AC54321");

        // Create Project

        Project p1 = new Project(301, "Payroll System", "TechCorp", 1500);

        // Create Timesheet for JEGANATH

        Timesheet t1 = new Timesheet(401, e1.getEmpId(), "2025-09-01", p1);

        t1.logHours("2025-09-01", 8);
        t1.logHours("2025-09-02", 9);
        t1.logHours("2025-09-03", 10);
        t1.logHours("2025-09-04", 8);
        t1.logHours("2025-09-05", 7);

        // Print Entries

        t1.printEntries();

        // Manager Approves

        m1.approveTimesheet(t1);

        // Payroll Service

        PayrollService ps = new PayrollService();

        ps.computePay(e1, t1);
        ps.computePay(m1, t1);

        // Project Report

        ps.projectReport(t1);

    }

}
```

PROGRAM EXECUTION FLOW:

1. Create Alice (Employee) and Bob (Manager).
2. Create Project → "Payroll System" for client TechCorp.
3. Alice logs hours for 5 days (8+9+10+8+7 = 42 hrs).
4. Alice's timesheet is initially PENDING.

5. Manager Bob approves the timesheet.
6. PayrollService computes payslips:
 - JEGANATH $\rightarrow 42 \text{ hrs} \times ₹500 = ₹21,000$
 - MONISH $\rightarrow 42 \text{ hrs} \times ₹800 = 33,600 + 2 \text{ hrs overtime} \times ₹400 = ₹34,400$
7. Project report shows hours and billable amount.

OUTPUT:

```
Timesheet Entries for Payroll System:
2025-09-03 -> 10 hrs
2025-09-02 -> 9 hrs
2025-09-01 -> 8 hrs
2025-09-05 -> 7 hrs
2025-09-04 -> 8 hrs
Manager approved Timesheet ID: 401

---- Payslip ----
Employee: JEGANATH
Role: Developer
Project: Payroll System
Total Hours: 42
Pay Amount: ₹21000.0
Bank Account: AC12345
-----

---- Payslip ----
Employee: MONISH
Role: Manager
Project: Payroll System
Total Hours: 42
Pay Amount: ₹34400.0
Bank Account: AC54321
-----

Project Report for Payroll System
Client: TechCorp
Total Hours: 42
Billable Amount: ₹63000.0
-----
```

KEY FEATURES IN THIS CODE:

1. **Encapsulation** $\rightarrow$ private fields + getters/setters in Timesheet, Employee, Project.
2. **Inheritance** $\rightarrow$ Manager extends Employee.
3. **Method Overloading** $\rightarrow$ logHours(date, hours) & logHours(date, start, end).
4. **Method Overriding** $\rightarrow$ computePay() in Manager.
5. **Polymorphism** $\rightarrow$ PayrollService.computePay(Employee emp, Timesheet sheet) works for both Employee & Manager objects.
6. **Project-wise Reporting** $\rightarrow$ PayrollService.projectReport() prints hours & billable amount.

1. PROBLEM UNDERSTANDING:

Organizations need a way to:

- Log employee work hours on projects.
- Approve/reject timesheets (managers).
- Compute payroll (salaries/wages based on hours).
- Generate payslips and reports.

A console-based application is a good starting point because it:

- Focuses on logic instead of UI.
- Demonstrates OOP principles clearly.

2. CORE CLASSES (OOP DESIGN):

(a) Employee Class:

1. Represents any worker in the company.
2. Attributes: empId, name, role, hourlyRate, bankAccount.
3. Behaviors (methods):
 - getters/setters() → encapsulation.
 - computePay(totalHours) → base pay calculation.
 - toString() → display details.

❖ Concept Used → *Encapsulation* (private/protected variables with controlled access).

(b) Manager Class (Inheritance)

1. A manager is also an employee but has extra responsibilities.
2. Inheritance: Manager extends Employee.
3. Adds ability to:
 - Approve timesheets → approveTimesheet().
 - Compute pay differently (with overtime/bonus) → computePay() overriding.

❖ Concept Used → *Inheritance + Method Overriding*.

(c) Project Class

1. Represents a project employees work on.
2. Attributes: projectId, name, client, billableRate.
3. Used to track effort and cost per project.

(d) Timesheet Class

1. Holds records of hours worked by an employee in a week.
2. Attributes: sheetId, empId, weekStart, state (PENDING/APPROVED), entries.
3. Behaviors:

- logHours(date, hours) → direct logging.
- logHours(date, startTime, endTime) → method overloading.
- getTotalHours() → sum up all hours.
- printEntries() → display all logged work.

❖ Concept Used → *Method Overloading* (same method name, different parameters).

(e) PayrollService Class

1. Service layer that uses Employee & Timesheet data.
2. Methods:
 - computePay() → calculate salary.
 - generateSlip() → print payslip.
 - Future: projectReport() → hours by project.

❖ Concept Used → *Polymorphism* (method works differently depending on if input is Employee or Manager).

3. OOP CONCEPTS DEMONSTRATED:

◆ Encapsulation:

1. Employee's bank account & salary details are private.
2. Access given only through getters/setters.
3. Protects sensitive data.

◆ Inheritance:

1. Manager inherits Employee.
2. Code reuse → Manager doesn't need to redefine common things (empId, name, etc.).

◆ Method Overloading:

1. logHours(date, hours) vs. logHours(date, start, end)
2. Same method name but different parameter lists.

◆ Method Overriding:

1. Employee → normal computePay().
2. Manager → overridden computePay() with overtime/bonus logic.

◆ Polymorphism:

Employee emp = new Manager(...);

1. Even though emp is declared as Employee, at runtime the Manager version of computePay() executes.

2. This is runtime polymorphism (dynamic dispatch).

4. FLOW OF THE APPLICATION:

1. Employee Creation:

- HR/Manager creates employee and project details.

2. Timesheet Logging:

- Employee logs hours daily → stored in Timesheet.

3. Submission & Approval:

- Timesheet is initially PENDING.
- Manager approves it → APPROVED.

4. Payroll Computation:

- PayrollService checks if the sheet is approved.
- If approved → compute pay using employee's hourly rate.
- If Manager → overtime/bonus considered.

5. Payslip Generation:

- Employee name, role, hours worked, pay amount, bank account printed.

5. EXAMPLE SCENARIO:

1. Alice (Developer, ₹500/hr) works 42 hrs in a week.
2. Bob (Manager, ₹800/hr) approves her timesheet.
3. PayrollService calculates:
 - Alice → $42 \times 500 = ₹21,000$.
 - Bob → if he worked 45 hrs → $40 \times 800 + 5 \times 800 \times 1.5 = ₹40,000$.
4. Payslips are printed to console.

BENEFITS OF OOP :

- **Reusable code** → Employee base class reused for Manager.
- **Maintainability** → If pay rules change, update only computePay().
- **Extensibility** → Can add Contractor or Intern class with custom pay easily.
- **Security** → Bank details & payroll figures hidden using encapsulation.

CONCLUSION:

- ❖ The console-based Timesheet & Payroll Application successfully demonstrates the use of Object-Oriented Programming (OOP) concepts in Java. The project enables employees to log working hours, managers to approve timesheets, and the payroll service to compute accurate payments and generate payslips.
 - ✓ Key OOP principles are applied as follows:
 - Encapsulation protects sensitive data such as payroll figures and timesheet states.
 - Inheritance allows the Manager class to extend Employee with additional approval rights.
 - Polymorphism enables handling both employees and managers under a common reference type.
 - Method Overloading is shown in the logHours() method (by hours or by start–end time).
 - Method Overriding is demonstrated in the computePay() method, where managers have overtime rules.
 - The application not only automates payroll processing but also ensures scalability for handling multiple employees, projects, and timesheets. This makes it a practical foundation for real-world payroll management systems.

GITHUB REPOSITORY LINK:

<https://github.com/jeganath047-ux/TimesheetPayrollApp-Java>